

Deep Learning

Assignment 3

Write a program to implement k-Nearest Neighbour algorithm to classify the iris data set. Print both correct and wrong predictions. Python ML library classes can be used for this problem.

Name: Rohit Kalaskar

Roll No: 24

PRN: 12310106

Branch: CS AI - B

```
# Import required Python libraries
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
```

```
df = pd.read_csv('customer_segmentation_mixed.csv')
```

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 350 entries, 0 to 349
Data columns (total 6 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Age                   350 non-null   int64
1   Annual_Income         350 non-null   int64
2   Spending_Score        350 non-null   int64
3   Monthly_Visits        350 non-null   int64
4   Gender                 350 non-null   object
5   Preferred_Product     350 non-null   object
dtypes: int64(4), object(2)
memory usage: 16.5+ KB
```

```
df.describe()
```

	Age	Annual_Income	Spending_Score	Monthly_Visits	
count	350.000000	350.000000	350.000000	350.000000	
mean	41.202857	71213.274286	49.571429	9.522857	
std	13.422450	29311.239749	29.602801	5.584290	
min	18.000000	20412.000000	1.000000	1.000000	
25%	30.000000	45981.750000	24.000000	5.000000	
50%	42.000000	71990.500000	51.000000	9.000000	
75%	52.000000	97945.500000	75.000000	14.000000	
max	64.000000	119835.000000	99.000000	19.000000	

```
df.head()
```

	Age	Annual_Income	Spending_Score	Monthly_Visits	Gender	Preferred_Product
0	56	26102	85	6	Female	Clothing
1	46	70336	75	18	Male	Home
2	32	105314	36	10	Male	Groceries
3	60	46641	99	5	Female	Electronics
4	25	54584	19	10	Male	Electronics

Next steps: [Generate code with df](#) [New interactive sheet](#)

```
print("\nMissing values:\n", df.isnull().sum())
```

```
Missing values:
Age                0
Annual_Income      0
Spending_Score     0
Monthly_Visits     0
Gender             0
Preferred_Product  0
dtype: int64
```

```
X = df.drop('Preferred_Product', axis=1)
y = df['Preferred_Product']
```

```
num_cols = ['Age', 'Annual_Income', 'Spending_Score', 'Monthly_Visits']
cat_cols = ['Gender']
```

```
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder, StandardScaler

numeric_transformer = Pipeline(steps=[
    ('scaler', StandardScaler())
])

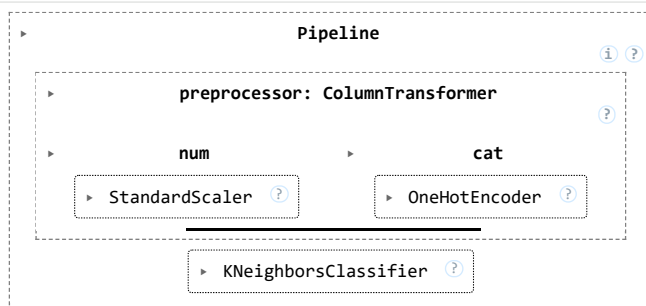
categorical_transformer = Pipeline(steps=[
    ('onehot', OneHotEncoder(handle_unknown='ignore', drop='first'))
])

preprocessor = ColumnTransformer(
    transformers=[
        ('num', numeric_transformer, num_cols),
        ('cat', categorical_transformer, cat_cols)
    ]
)
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

```
knn_model = Pipeline(steps=[
    ('preprocessor', preprocessor),
    ('classifier', KNeighborsClassifier(n_neighbors=5))
])
```

```
knn_model.fit(X_train, y_train)
```



```
y_pred = knn_model.predict(X_test)
print("\nPrediction Results:\n")

for actual, predicted in zip(y_test, y_pred):
    if actual == predicted:
        print(f"CORRECT → Actual: {actual}, Predicted: {predicted}")
    else:
        print(f"WRONG → Actual: {actual}, Predicted: {predicted}")
```

[Show hidden output](#)

```
accuracy = accuracy_score(y_test, y_pred)
print(f"\nModel Accuracy: {accuracy * 100:.2f}%")
```

Model Accuracy: 10.00%

```
print("\nClassification Report:\n")
print(classification_report(y_test, y_pred))
```

Classification Report:

	precision	recall	f1-score	support
Clothing	0.04	0.10	0.05	10
Electronics	0.12	0.22	0.15	9
Groceries	0.14	0.06	0.09	16
Home	0.18	0.12	0.14	17
Sports	0.14	0.06	0.08	18
accuracy			0.10	70
macro avg	0.12	0.11	0.10	70
weighted avg	0.13	0.10	0.10	70

```
cm = confusion_matrix(y_test, y_pred)

classes = np.unique(y_test)

print("Confusion Matrix:\n", cm)

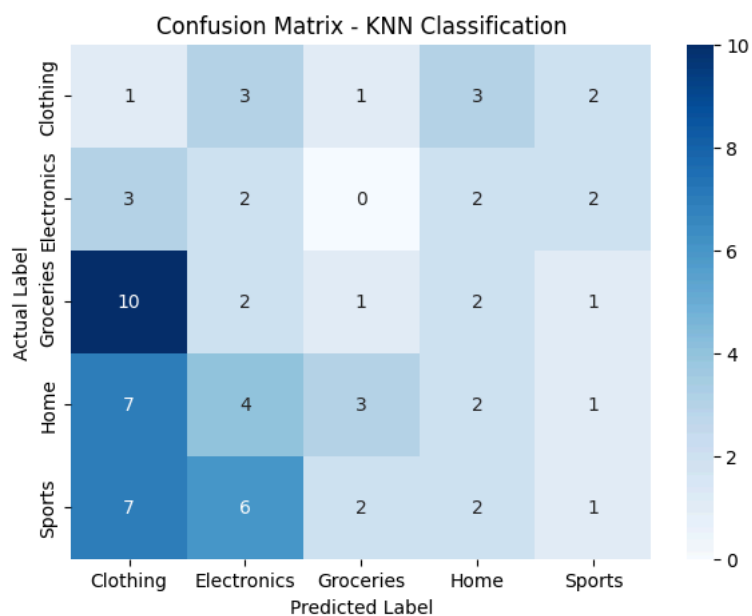
print("\nTP, FP, FN, TN for each class:\n")

for i, cls in enumerate(classes):
    TP = cm[i, i]
    FP = cm[:, i].sum() - TP
    FN = cm[i, :].sum() - TP
    TN = cm.sum() - (TP + FP + FN)

    print(f"Class: {cls}")
    print(f"TP: {TP}")
    print(f"FP: {FP}")
    print(f"FN: {FN}")
    print(f"TN: {TN}\n")
```

[Show hidden output](#)

```
plt.figure(figsize=(7,5))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=np.unique(y),
            yticklabels=np.unique(y))
plt.xlabel("Predicted Label")
plt.ylabel("Actual Label")
plt.title("Confusion Matrix - KNN Classification")
plt.show()
```



Start coding or [generate](#) with AI.

